CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment 10

Student Name: Aaditya Anand **UID: 23BCS10446**
Branch: BE CSE **Section/Group: KRG 3A**
Semester: 6th **Date of Performance: 20/04/26**
Subject Name: Full Stack Development **Subject Code: 23CSH-309**

Aim:

To design and implement a secure, scalable full-stack ride-booking application using Spring Boot by integrating Spring Security, JWT-based authentication, Role-Based Access Control (RBAC), and Spring Data JPA for efficient database management, along with a responsive frontend using React.

Objective:

1. Implement backend security using Spring Security with custom filter chains and JWT-based authentication.
2. Enable authentication mechanisms such as OAuth (e.g., Google Login) for secure third-party login integration.
3. Apply Role-Based Access Control (RBAC) to restrict and protect API endpoints based on user roles (USER, DRIVER, ADMIN).
4. Optimize database interactions using Spring Data JPA with efficient queries, pagination, and performance techniques.
5. Integrate the secure backend with a responsive frontend built using React, ensuring proper CORS configuration for seamless communication.

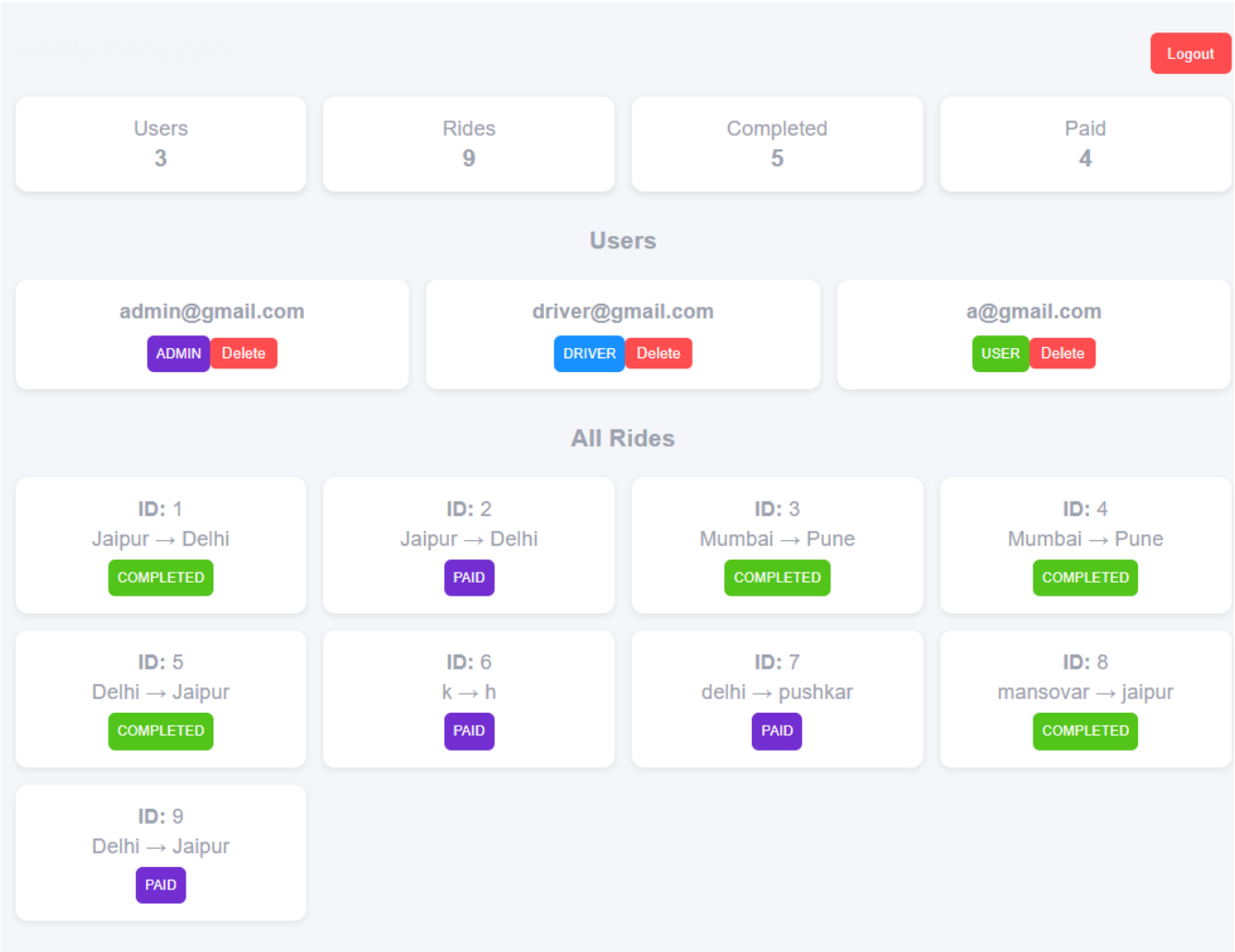
Implementation/Code:

AdminController.java

```
package com.ecotrack.livepoll.controller;
import com.ecotrack.livepoll.auth.AppUserPrincipal;
import com.ecotrack.livepoll.dto.PollDtos;
import com.ecotrack.livepoll.service.PollService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
```

```
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/admin/polls")
@PreAuthorize("hasRole('ADMIN')") // 🔒 Only ADMIN can access
public class AdminPollController {
    private final PollService pollService;
    public AdminPollController(PollService pollService) {
        this.pollService = pollService;
    }
    // Create Poll
    @PostMapping
    public ResponseEntity<PollDtos.PollResponse> createPoll(
            @Valid @RequestBody PollDtos.CreatePollRequest request,
            @AuthenticationPrincipal AppUserPrincipal principal
    ) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(pollService.createPoll(request, principal.getUsername()));
    }
    //  Close Poll
    @PostMapping("/{id}/close")
    public ResponseEntity<PollDtos.PollResponse> closePoll(@PathVariable Long id) {
        return ResponseEntity.ok(pollService.closePoll(id));
    }
    //  Delete Poll
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletePoll(@PathVariable Long id) {
        pollService.deletePoll(id);
        return ResponseEntity.noContent().build();
    }
}
```

Output:



Learning Outcome:

- Learned how Spring Security secures backend APIs using filters and authentication mechanisms.
- Clearly understood the difference between authentication (who you are) and authorization (what you can access).
- Implemented JWT-based authentication for secure login and request validation.
- Gained hands-on experience with Role-Based Access Control (RBAC) to protect endpoints (USER, DRIVER, ADMIN).
- Learned how to integrate a React frontend with a secured backend using proper CORS configuration.

- Understood the complete request flow through security filters (JWT filter, authentication, authorization).
- Improved debugging and analysis skills for secured full-stack applications.